

Estruturas de Dados

Dicionários e prioridade

REALIZAR APÓS posição 21: Estruturas de Dados: 21 Heaps e HeapSort

Posições na playlist 18 a 21 | 120 minutos | 100 pontos

Curso principal: Estruturas de Dados |

<https://www.youtube.com/watch?v=0hT3EKGhbpl&list=PLndfcZyvAqbofQI2kLLdeWWjCcPIOPnrW>

Conteúdo verificável: Dicionário; análise amortizada; prioridade; heaps; heapsort

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Dicionário

(a) Especifique inserir, consultar, remover num dicionário com chaves únicas. (b) Projete implementação simples e discuta custo. (c) Compare dicionário ordenado e não ordenado para consultas de intervalo sem assumir um algoritmo de hash específico.

Desenvolvimento / justificativa / esquema:

Análise amortizada

Uma pilha em vetor dobra capacidade quando fica cheia. (a) Conte cópias numa sequência de n inserções desde capacidade 1. (b) Mostre custo amortizado constante. (c) Diferencie custo amortizado de custo médio probabilístico.

Desenvolvimento / justificativa / esquema:

Heap máximo

(a) Construa heap máximo a partir de [3, 1, 6, 5, 2, 4] por heapify de baixo para cima. (b) Extraia máximo e mostre restauração. (c) Dê custos de construir, inserir e extrair.

Desenvolvimento / justificativa / esquema:

Fila de prioridade e heapsort

(a) Explique escolha de heap para tarefas com prioridades mutáveis e quais dados extras permitem localizar uma tarefa. (b) Descreva heapsort in-place crescente. (c) Analise tempo, espaço auxiliar e estabilidade.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Dicionário (15 pontos)

Inserir deve definir se atualiza chave existente; consultar informa ausência; remover tem comportamento documentado para chave ausente. Lista simples faz operações de busca $O(n)$; BST custa $O(h)$, balanceada $O(\log n)$. Para intervalo, estrutura ordenada facilita localizar início e percorrer valores; uma estrutura não ordenada pode exigir varredura completa. O contrato não determina sozinho a implementação ou complexidade.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Análise amortizada (25 pontos)

Realocações copiam $1+2+4+\dots$ até potência menor que n , soma $< 2n$. Inserções propriamente ditas custam n , então total $O(n)$, amortizado $O(1)$. Uma inserção que realoca custa $\Theta(n)$, mas a sequência dilui esse custo. Não há distribuição aleatória pressuposta: amortização é garantia sobre sequência de operações, não média sobre entradas sorteadas.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Heap máximo (25 pontos)

Uma construção resulta $[6, 5, 4, 1, 2, 3]$. Extração troca último 3 para raiz e reduz tamanho; sift-down troca 3 com 5, resultando $[5, 3, 4, 1, 2]$. Construção $O(n)$, inserção e extração $O(\log n)$. Heap impõe pai $\geq$ filhos, não ordenação completa entre irmãos. Aceitar outra disposição que tenha os mesmos elementos e satisfaça a propriedade.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Fila de prioridade e heapsort (35 pontos)

Heap permite extrair maior/menor prioridade em $O(\log n)$; para atualizar uma tarefa específica eficientemente, manter mapa identificador $\rightarrow$ posição e atualizá-lo nas trocas. Heapsort usa heap máximo; troca raiz com último elemento da região ativa, reduz região e restaura heap. Tempo $O(n \log n)$, espaço auxiliar $O(1)$ com implementação iterativa. Geralmente não é estável; trocas distantes podem inverter iguais. Não confundir o vetor ordenado final com o heap ativo intermediário.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.